

Lab 1

In this lab, you'll be parallelizing the given code for a heat diffusion simulator.

Heat diffusion in parallel

Begin by downloading the given code. Compile and run `Main.java` inside the `serialHeat` package. Run the program, drawing a scenario and then running a simulation for 5000 timesteps. As shown in class, I like scenarios with some heaters near an insulator. You want a scenario that is simple enough that you'll be able to approximately duplicate it. (The specific contents of the scenario don't heavily affect the running time, but our running time comparisons will be more legitimate if we're running the same thing...) Make a note of the running time (printed after the 5000th time step).

The focus of our changes is the `diffuse` method in `DataArray.java`. This method is a doubly-nested loop that calls `diffusePixel` to update each pixel in the simulation during each time step of the simulation. This is the most time-consuming part of the simulation and its parts are also independent so it makes sense to focus our parallelization efforts here. We are going to split the iterations in `diffuse` between different tasks that can run in parallel.

To do this, let's use the fork-join framework. The file `SumArray.java` contains the example of summing array values that we worked with in class. You'll want to adapt this by creating an extension of `RecursiveTask` to parallelize the `y` loop in `diffuse`. Your method won't have a meaningful return value so just make the `compute` method return 0.¹ The threshold does not seem to have a large effect on the running time. Follow the example of `SumArray` and just split the `y` loop; the whole `x` loop goes into the base case of the recursion. It is easiest if you create your new class inside the definition of `DataArray` (i.e. make it an inner class) so that it will have access to all the attributes of `DataArray`. If you do this, the only attributes of the class will be `low` and `hi`. If you make the class outside the definition of `DataArray`, then you'll have to pass it `X`, `isInsulator`, `isHeater`, `currArr`, and `nextArr`.

After you've parallelized the program, run it and see how long it takes for the same scenario that you drew originally.

¹There is a different class `RecursiveAction` that you could use instead, but I'm opting for `RecursiveTask` since that's what we used in class.